


**UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA**

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE

CORSO DI STUDIO TRIENNALE IN
**INFORMATICA**

# Sviluppo di uno strumento di migrazione dati Infor LN

**Relatore:**
Prof. Massazza Paolo

**Tesi di Laurea di:**
Gabriele Caretti
Matricola 756564

**Anno accademico:**
2025/2026

# Indice


**1 Introduzione** **1**

1.1 Contesto Aziendale . . . . . 1
1.2 Obiettivi del Progetto . . . . . 1
1.3 Metodologia e Strumenti . . . . . 2
1.4 Contesto Tecnologico e Curva di Apprendimento . . . . . 2
1.5 Struttura della Tesi . . . . . 3

**2 Stato Attuale** **5**

2.1 Sistemi ERP . . . . . 5
2.1.1 Evoluzione dei Sistemi Legacy al Cloud . . . . . 5
2.1.2 Architetture Multitenant e Segregazione dei Dati . . . . 5
2.2 Infor LN e CloudSuite Enterprise . . . . . 6
2.2.1 Linguaggio Baan C: 3GL e 4GL . . . . . 8
2.2.2 Infor Exchange e Strumenti di Import/Export . . . . . 8
2.3 Migrazione Dati nei Sistemi ERP . . . . . 8
2.3.1 Il Paradigma ETL (Extract, Transform, Load) . . . . . 8
2.3.2 Sfide e Problematiche Comuni . . . . . 10
2.3.3 Necessità di Strumenti di Automazione e Generazione Dinamica . . . . . 10

**3 Analisi dei Requisiti** **11**

3.1 Scenario di Riferimento . . . . . 11
3.1.1 Strumento di Migrazione Esistente . . . . . 11
3.2 Requisiti Funzionali . . . . . 11
3.2.1 Sviluppo dell'Importer Infor Exchange . . . . . 11
3.2.2 Estensibilità con Funzioni Ad-hoc . . . . . 12
3.2.3 Integrazione con lo Strumento Documentale . . . . . 12
3.3 Requisiti Non Funzionali . . . . . 12
3.3.1 Manutenibilità ed Estensibilità . . . . . 12
3.3.2 Sicurezza e Data Integrity in Multitenancy . . . . . 12
3.4 Modello dei Dati . . . . . 13

3.4.1 Analisi Strutture Dati Coinvolte . . . . . 13

**4 Architettura** **16**
4.1 Panoramica dell'Architettura di Sistema . . . . . 16
4.1.1 Modello Dati e Dizionario . . . . . 16
4.2 Progettazione del Software . . . . . 18
4.2.1 Design dell'Importer e del Parser dei Dati . . . . . 18
4.2.2 Generazione del Codice . . . . . 19
4.2.3 Modulo di Integrazione Ad-hoc e Callbacks . . . . . 19
4.3 Scelte Tecnologiche . . . . . 19
4.3.1 Linguaggi e Framework . . . . . 19
4.3.2 Librerie di Supporto e Strumenti di Build . . . . . 19

**5 Implementazione** **21**
5.1 Sviluppo dell'Importer Infor Exchange . . . . . 21
5.1.1 Gestione dei Formati e Parsing . . . . . 21
5.1.2 Ricostruzione delle Definizioni dei File da Migrare . . . . 22
5.2 Implementazione del Generatore di Codice . . . . . 24
5.2.1 Creazione Dinamica del Codice . . . . . 24
5.2.2 Funzioni di Trasformazione Ad-hoc . . . . . 26
5.3 Interfacce Utente (Sessioni) . . . . . 26
5.3.1 Esecuzione e Monitoraggio . . . . . 27
5.3.2 Librerie (DLL) e Data Access Layer (DAL) . . . . . 29
5.3.3 Tabelle di Sistema e Sessioni Operative . . . . . 29
5.4 Integrazione Documentale . . . . . 31
5.4.1 Esportazione delle Specifiche di Migrazione . . . . . 31
5.5 Ciclo di Vita del Dato . . . . . 32

**6 Testing e Validazione** **33**
6.1 Strategia di Testing . . . . . 33
6.1.1 Unit Testing sui Moduli di Parsing . . . . . 33
6.1.2 Test di Integrazione del Flusso di Migrazione . . . . . 33
6.2 Verifica dell'Integrità dei Dati . . . . . 34
6.2.1 Matching tra Dati Sorgente (Legacy) e Destinazione (Cloud) 34

**7 Conclusioni** **35**
7.1 Obiettivi Raggiunti e Risultati Ottenuti . . . . . 35

7.2 Sfide Incontrate . . . . . 35
7.3 Sviluppi Futuri . . . . . 36

**A Dettaglio della Struttura delle Tabelle** **37**
A.1 Tabelle di Configurazione e Metadati . . . . . 37
A.2 Tabelle di Mapping e Logica . . . . . 39
A.3 Tabelle di Esecuzione e Log . . . . . 41

**Bibliografia** **43**

# Capitolo 1

# Introduzione

## 1.1 Contesto Aziendale

Il progetto di tesi si propone di sviluppare uno strumento di migrazione dati per Infor LN [6], un sistema *ERP* (*Enterprise Resource Planning*) industriale utilizzato a livello internazionale.

L'obiettivo è agevolare il processo di migrazione da sistemi *legacy* a Infor LN, garantendo l'integrità e la coerenza dei dati in ambienti *multitenant* [1, 2].

In primis, questo strumento sarà sviluppato all'interno dell'azienda **AlteaIN**, azienda parte del gruppo di Altea Federation e *partner* ufficiale Infor in Italia. AlteaIN è specializzata nella consulenza informatica, e si occupat di supportare le aziende del comparto manifatturiero e logistico nei loro processi di trasformazione digitale e innovazione tecnologica.

## 1.2 Obiettivi del Progetto

Il lavoro si concentrerà sull'analisi dei requisiti, la progettazione dell'architettura del software, la stesura della documentazione e lo sviluppo di un prototipo funzionante.

In particolare, lo strumento sarà in grado di generare dinamicamente codice, estendibile con funzioni ad hoc, per migrare dati su sistemi *Cloud Multitenant*, con un focus sulla Infor *Cloudsuite Enterprise*. Questa generazione permetterà di abbattere drasticamente i tempi di sviluppo di *script customizzati* per ciascuna migrazione, riducendo i classici errori manuali e aumentandone l'efficienza.

Inoltre, è richiesto lo sviluppo di un *importer* capace di interpretare il formato proprietario di *import/export* di Infor *Exchange* al fine di ricreare automaticamente le definizioni fisiche dei file dall'origine da migrare. L'intero sistema andrà ad interfacciarsi e integrarsi con lo strumento aziendale già esistente per la produzione di specifiche di migrazione e la relativa esportazione documentale.

## 1.3 Metodologia e Strumenti

Essendo il *Migration Tool* destinato a diventare uno strumento *core* per i consulenti AlteaIN, lo sviluppo ha richiesto alti requisiti di qualità, seguendo le *best practice* dell'ingegneria del software, sotto la supervisione del tutor aziendale.

Durante i mesi di tirocinio, le attività sono state gestite ispirandosi ai princìpi *Agile*. Il lavoro è stato scandito da momenti di confronto regolari (*stand-up meeting* e revisioni periodiche con il tutor aziendale) che hanno permesso di allineare costantemente lo sviluppo ai requisiti emergenti precedentemente menzionati e alle stringenti regole imposte dall'architettura Infor.

Per il tracciamento delle attività, l'analisi dei requisiti e la stesura della documentazione tecnica, si è fatto affidamento su strumenti di collaborazione aziendale, garantendo così una continua sinergia tra la figura dello sviluppatore (il sottoscritto) e i consulenti funzionali. Quest'ultimi, in quanto futuri utenti finali del *tool*, hanno giocato un ruolo chiave nel definire gli *Use Case* e nel testare le *release* intermedie del generatore di codice.

## 1.4 Contesto Tecnologico e Curva di Apprendimento

Uno degli aspetti più sfidanti e formativi del tirocinio, preliminare alla fase di sviluppo vera e propria, è stato l'inserimento in un ecosistema software proprietario ed estremamente verticalizzato. A differenza dei linguaggi *open-source* o dei framework convenzionali largamente studiati durante il percorso accademico (come Java o framework web), l'ambiente Infor si basa su un'architettura consolidata denominata *Infor Tools*.

Il linguaggio di programmazione impiegato è il **Baan C** (nelle sue versioni 3GL e 4GL), un linguaggio procedurale ed event-driven strettamente legato al dizionario dati (*Data Dictionary*) del database sottostante (che può essere di diversi tipi, a noi non noti per motivi riservati). Questa peculiarità ha richiesto un importante periodo iniziale di *training* e studio della documentazione ufficiale (come *progguide* e videolezioni registrate create per altri colleghi e clienti), necessario per padroneggiare concetti avanzati come le *Dynamic Link Libraries* (DLL) proprietarie, i *Data Access Layer* (DAL), la gestione della memoria in un contesto *Enterprise* e l'utilizzo degli strumenti dedicati. Solo a valle di questa significativa curva di apprendimento è stato possibile ideare e implementare i moduli architetturali che compongono il *Migration Tool*.

Tra questi elementi si possono citare:

- **Architettura**: Infor segue un'architettura a più livelli, da quelli fisici, database, a quelli di comunicazione con l'esterno tramite middleware ION API (in fase di transizione da XML con SOAP a REST), includendo web services e nuovi strumenti.
- **Strumenti di sviluppo**: Strumenti di sviluppo come IDE proprietari (basati su IDE più famosi come *Eclipse*) LN Studio, BECS e interfaccia web LNUI.
- **Librerie e moduli**: L'ecosistema Infor è composto da numerose librerie e moduli preesistenti, che vanno dalle utility di stringa alle librerie di accesso ai dati, fino a moduli specifici per la gestione dei processi di business.

## 1.5 Struttura della Tesi

L'elaborato si articola nel seguente modo:

- **Capitolo 2**: stato attuale degli ERP moderni, architetture cloud multitenant e sfide di migrazione dati.
- **Capitolo 3**: analisi dello scenario di partenza e dei requisiti funzionali e non funzionali.
- **Capitolo 4**: architettura del sistema, design e scelte tecnologiche (come Importer e Generatore codice).

- **Capitolo 5**: descrizione dell'implementazione concreta del progetto.
- **Capitolo 6**: testing, inclusi elementi come la validazione dell'integrità dei dati.
- **Capitolo 7**: conclusioni, traguardi raggiunti e possibili sviluppi futuri.

# Capitolo 2

# Stato Attuale

## 2.1 Sistemi ERP

I sistemi *ERP* (*Enterprise Resource Planning*) rappresentano un elemento essenziale del sistema gestionale aziendale [7], consentendo l'integrazione dei processi di *business* principali di un'azienda come: vendite, acquisti, gestione inventario, contabilità e personale.

### 2.1.1 Evoluzione dei Sistemi Legacy al Cloud

Nel corso degli anni, i sistemi *ERP* si sono evoluti partendo da architetture *on-premise legacy*, spesso caratterizzate da un elevato grado di personalizzazione (*customization*, in Infor LN definiti come `tx`) e importanti costi di manutenzione. Tuttavia, oggi si sta assistendo ad una transizione verso nuove soluzioni *Cloud*.

### 2.1.2 Architetture Multitenant e Segregazione dei Dati

Di fatto, il modello *Cloud Multitenant* oggi rappresenta l'architettura fondamentale degli *ERP* moderni [1]. A differenza dei tradizionali sistemi *on-premise* (definiti *single-tenant*), in cui ogni azienda possiede un'infrastruttura hardware e software fisicamente isolata, in un ambiente *multitenant* una singola istanza dell'applicativo e del database viene condivisa da molteplici organizzazioni clienti (i cosiddetti *tenant*, in un ambiente *cloud*).

Questo approccio offre molti vantaggi dal punto di vista dell'erogazione del servizio (scalabilità orizzontale, aggiornamenti continui e massivi, ottimizzazione delle risorse e abbassamento dei costi di manutenzione server), ma pone

grandi sfide architetturali. La principale problematica risiede nell'isolamento logico delle informazioni: essendo i dati fisicamente residenti nello stesso database, è vitale garantire una rigorosa divisione (segregazione) per evitare violazioni di sicurezza e fughe di dati (data leak) tra aziende ospitate sulla stessa piattaforma.

Nei sistemi come Infor *CloudSuite Enterprise*, la segregazione dei dati viene applicata a più livelli. A livello di infrastruttura *Cloud*, l'isolamento del *tenant* è garantito da chiavi di crittografia specifiche e partizionamenti logici. A livello di applicativo, la storicizzazione e l'isolamento avvengono tramite l'uso sistematico di identificativi aziendali (spesso indicati come *Company Number* o codici azienda) che vengono inseriti implicitamente dal *framework* in ogni query al database.

Da questa architettura deriva la necessità che ogni processo di migrazione operi secondo vincoli di integrità estremamente rigidi [8]. Qualsiasi inserimento massivo di dati deve necessariamente passare attraverso strati intermedi di validazione (come i *Data Access Layer*) che assicurino che il dato venga assegnato non solo alla tabella corretta, ma anche e soprattutto all'identificativo del *tenant* di destinazione, senza possibilità di errore o sovrascrittura accidentale errata.

## 2.2 Infor LN e CloudSuite Enterprise

Infor LN, nato con il nome di Baan *ERP*, è uno dei principali *ERP* nel settore manifatturiero industriale [6] a livello internazionale. Viene integrato all'interno della Infor *CloudSuite Enterprise*, il quale offre funzionalità per la gestione della *supply chain* e produzione [2].

L'interfaccia utente Web (nota come `LNUI`) permette agli utenti finali di navigare tra le diverse sessioni tramite *menu* contestuali o ricerca diretta (Figura 2.1).

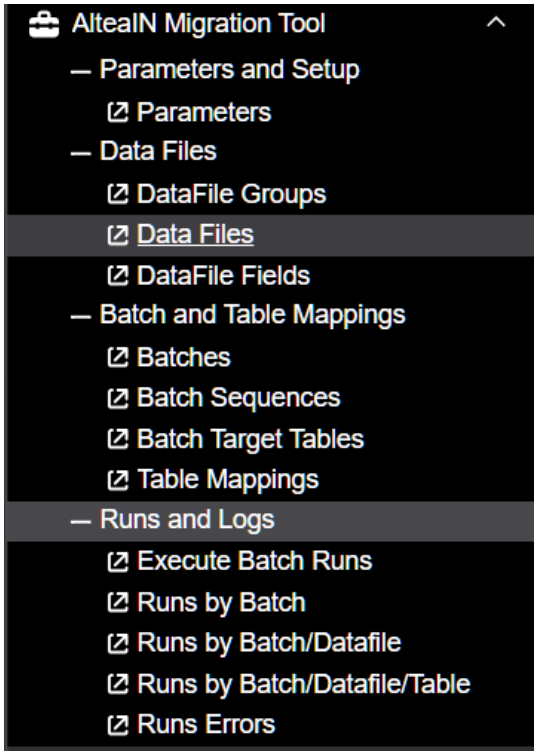



Figura 2.1: Esempio di interfaccia Web Infor LN (LNUI) con elenco sessioni del Migration Tool.

Infor LN è basato su un'architettura che separa il *framework* tecnologico (gli Infor Tools) dalla logica di *business*. Ogni applicazione è composta da diversi oggetti:

- **Tabelle (*Tables*):** I dati fisici, definiti tramite *Data Dictionary*, con domini standard o *custom*.
- **Sessioni (*Sessions*):** Le interfacce utente (da qui definite come Sessioni) attraverso cui gli operatori accedono, visualizzano e interagiscono con i dati.
- **DAL (*Data Access Layer*):** L'elemento che si occupa di garantire l'integrità dei dati e delle tabelle, indipendentemente dalle Sessioni [4].
- **Domini (*Domains*):** Definiscono i tipi di dati, con regole di validazione e formati, che possono essere standard o personalizzati (*custom*).
- **Etichette (*Labels*):** Utilizzate per la localizzazione e la visualizzazione dei campi nelle Sessioni, con supporto multilingue (es. 02 corrisponde alla lingua inglese, e 06 alla lingua italiano).

### 2.2.1 Linguaggio Baan C: 3GL e 4GL

Per lo sviluppo e personalizzazione degli *script* e librerie, Infor LN utilizza un linguaggio proprietario, di nome **Baan C** [3].

Esso si divide a sua volta in:

- **4GL (*Fourth-Generation Language*):** La versione più recente e moderna. Infor LN Tools genera automaticamente operazioni standard di Sessione e database (CRUD). In 4GL non è necessario scrivere codice per gestire il salvataggio o il caricamento di un record, ma ci si concentra sulla gestione degli *eventi* legati ai componenti della Sessione o del database (tramite *DAL* [5]).
- **3GL (*Third-Generation Language*):** Precedente a 4GL, e totalmente integrato con esso, viene utilizzato per la gestione di logiche di *business* complesse. Permette un approccio più procedurale e a basso livello per implementare funzioni matematiche, algoritmi personalizzati, *DLL* (*Dynamic Link Libraries*) proprietarie, funzioni e routine di manipolazione stringhe o file, nonchè di estensioni (*extensions*) che rappresentano in alcuni casi, l'unico modo di interagire e personalizzare tabelle e procedure standard.

### 2.2.2 Infor Exchange e Strumenti di Import/Export

Per lo scambio di informazioni, Infor fornisce un modulo denominato *Exchange* [9]. Si tratta di una piattaforma per l'importazione e l'esportazione di file (es. CSV, XML, testo a larghezza fissa) mappando righe e colonne sui *Datafiles* e sui campi del database di LN .

Spesso sono necessarie delle personalizzazioni delle procedure di migrazione tramite Baan C, che quindi risulta essenziale quando le regole di mappatura superano la complessità gestibile nativamente dall'*Exchange*.

## 2.3 Migrazione Dati nei Sistemi ERP

### 2.3.1 Il Paradigma ETL (Extract, Transform, Load)

Per capire appieno le dinamiche e le difficoltà che nascono dal processo di migrazione dei dati, è essenziale inquadrare tali operazioni all'interno del

paradigma architetturale *ETL* (*Extract, Transform, Load*). Questo modello teorico descrive le tre fasi fondamentali necessarie per prelevare dati da uno o più sistemi sorgenti (*source*), spesso eterogenei e obsoleti, per prepararli e infine inserirli in un sistema di destinazione (*target*).

Nello specifico, le tre fasi si articolano nel seguente modo:

- **Extract (Estrazione):** Rappresenta il processo di lettura e acquisizione delle informazioni dal sistema *legacy*. In questa fase i dati spesso si presentano in formati non strutturati o debolmente strutturati (come file CSV, esportazioni ASCII a larghezza fissa o vecchi archivi relazionali non normalizzati). La priorità in questa fase è estrarre il dato senza alterarne la semantica originale, gestendo le problematiche di codifica dei caratteri (ad esempio *UTF-8* contro vecchie codifiche *ISO*) e di formattazione.
- **Transform (Trasformazione):** È la fase più complessa e importante dell'intero paradigma. I dati grezzi appena estratti vengono analizzati, ripuliti e manipolati per adattarsi alle regole di *business* del nuovo *ERP*. Ciò include operazioni di *data cleansing* (rimozione dei duplicati o record corrotti), *data mapping* (associazione logica tra i campi del vecchio sistema e del nuovo), e *casting* dei tipi (ad esempio, la conversione di una stringa testuale di dati in un *timestamp* UTC).
- **Load (Caricamento):** L'ultima fase consiste nella scrittura fisica dei dati processati nel database *target*. In ambienti complessi, questa scrittura non avviene quasi mai tramite interrogazioni SQL dirette, bensì passando attraverso *API* o livelli di astrazione (come il DAL) che si occupano di avviare eventi aziendali, ricalcolare saldi e far scattare controlli di integrità referenziale.

Lo sviluppo di uno strumento software automatizzato (*tool*), come quello trattato in questa tesi, risponde all'esigenza di coprire tutte le fasi del paradigma *ETL* all'interno del medesimo ecosistema. L'obiettivo è superare le classiche frammentazioni dove sviluppatori diversi usano codici, linguaggi, e metodoligie diverse per le tre fasi, aumentando vertiginosamente la probabilità di errori semantici durante il trasferimento delle informazioni.

### 2.3.2 Sfide e Problematiche Comuni

La migrazione da un sistema *legacy* a un nuovo *ERP cloud* richiede un'attenta fase di analisi per evitare perdite o disallineamenti di informazioni storiche e *master data*.

Attualmente, molte migrazioni sono gestite manualmente, con l'integrazioni di *script* scritti *ad hoc*, con un alto livello di complessità e rischio d'errore. Per questo, nasce l'idea del Tool di Migrazione (*Migration Tool*) oggetto di questa tesi.

### 2.3.3 Necessità di Strumenti di Automazione e Generazione Dinamica

Per la migrazione verso il *Cloud*, non è più sostenibile un approccio basato esclusivamente su procedure di inserimento manuali o sullo sviluppo isolato di *script* monouso scritti *ad hoc* per ogni singola tabella aziendale. Questa pratica, oltre a comportare un ingente dispendio di ore lavorative per i consulenti, espone il progetto a continui errori umani derivanti dal copia-incolla delle logiche.

Inoltre, i moderni *ERP SaaS* richiedono strumenti robusti in grado di interfacciarsi con il database non solo in esecuzione *batch* locale, ma anche attraverso servizi *web* (*WebServices* o *API REST*) in scenari distribuiti. Il dato deve essere normalizzato, validato a livello applicativo e controllato prima ancora di essere introdotto al *database server*.

Emerge quindi una lacuna tra i vecchi processi operativi e le stringenti necessità dell'infrastruttura Infor *CloudSuite*. Diviene imperativo lo sviluppo di *framework* di migrazione che automatizzino la fase di scrittura del codice: partendo da mappature logiche definite dall'utente tramite interfacce grafiche (per quanto possibile), il *tool* deve essere in grado di generare a *runtime* le procedure di conversione. Solo eliminando la scrittura manuale degli script di mapping è possibile garantire la ripetibilità del processo, una diagnostica precisa degli errori e la totale tracciabilità di ogni singolo record rifiutato, requisito essenziale per la riuscita del progetto verso il nuovo sistema.

# Capitolo 3

# Analisi dei Requisiti

## 3.1 Scenario di Riferimento

### 3.1.1 Strumento di Migrazione Esistente

Negli ambienti *legacy*, come già menzionato nel capitolo precedente,il processo di migrazione dei dati [8] si basa su procedure manuali e *script ad hoc*.

L'operatore (noi) era costretto a sviluppare *script* personalizzati per ogni singola tabella e sessione, con lunghi tempi di realizzazione e grandi rischi inerenti alla trascrizione manuale delle logiche di *mapping*.

Inoltre, cambiamenti da parte di Infor e nuovi *standard* più restrittivi hanno reso le operazioni manuali molto più complesse [7]. Emerge quindi la necessità aziendale di innovare, ottimizzare e automatizzare il processo con un nuovo strumento, ossia l'integrazione di un *Migration Tool*.

## 3.2 Requisiti Funzionali

I requisiti funzionali definiscono i comportamenti attesi, ossia che cosa dovrebbe fare il nuovo strumento, così come stabilito con gli stakeholder.

### 3.2.1 Sviluppo dell'Importer Infor Exchange

Il sistema dovrà supportare l'*input* di file e strutture generate dallo strumento preesistente nativo di Infor *Exchange*.

Tali file presentano specifici *layout* (come formati testuali *ASCII* o delimitati) che dovranno essere interpretati per popolare il dizionario `txamg110` (*Datafiles*) e `txamg111` (*DataFile Fields*).

Alla base del *Migration Tool* ci sarà lo strumento generatore di codice. Partendo dalle tabelle di *mapping* (es. `txamg203`), lo strumento dovrà generare *script* di elaborazione e conversione, senza l'intervento manuale del programmatore.

### 3.2.2 Estensibilità con Funzioni Ad-hoc

Nonostante l'automazione, alcune regole di *legacy* di *business* non potranno essere legate a semplici assegnazioni.

Viene quindi la necessità di una funzionalità di *callback* per permettere, ad opera dello sviluppatore, l'inserimento di *script* personalizzati che verranno inseriti nel processo principale durante l'esecuzione *batch* (`txamg3200`).

### 3.2.3 Integrazione con lo Strumento Documentale

Per garantire l'allineamento con gli *standard* del progetto, il *Migration Tool* dovrà interfacciarsi con la documentazione aziendale, permettendo la produzione ed esportazione di specifiche tecniche di migrazione, *mappature*, eccezioni e *log* generati in fase di esecuzione (necessari per *debugging*).

## 3.3 Requisiti Non Funzionali

I requisiti non funzionali gestiscono i vincoli qualitativi e infrastrutturali del software.

### 3.3.1 Manutenibilità ed Estensibilità

Il *Migration Tool* dovrà essere interamente sviluppato in linguaggio Baan C rispettando il *framework* Infor Tools, così da garantirne la compatibilità con future *release* dell'*ERP* e manutenibilità semplificata per il comparto IT aziendale.

### 3.3.2 Sicurezza e Data Integrity in Multitenancy

Dato che il sistema tratterà dati di tipo *core* (come informazioni private o riservate) dell'azienda cliente in contesti *Cloud*, la sicurezza e la segregazione (isolamento) dei dati per i *Tenant* costituiscono priorità assolute e fondamentali.

Inoltre, il caricamento dovrà ottimizzare l'uso della memoria e delle sessioni, prediligendo una elaborazione a blocchi al fine di minimizzare i tempi morti generati dalla latenza di rete verso i server *CloudSuite* remoti.

## 3.4 Modello dei Dati

### 3.4.1 Analisi Strutture Dati Coinvolte

Per supportare in modo efficace il processo di migrazione, è stata definita un'architettura dati, basata su package *tx* (*custom*).

Il modello dati è suddiviso in diverse entità:

- **Parametrizzazione:**: tramite tabelle di configurazione come `txamg000` (AlteaIN Migration Tool Parameters).
- **Gestione dei Datafile:** tabelle come `txamg110` (Datafiles) e `txamg111` (DataFile Fields) permettono di definire le strutture dei file originali da importare.
- **Elaborazione dei Batch:** la configurazione e l'esecuzione tramite batch è regolata da tabelle come `txamg200` (Batch), `txamg201` (Batch Sequences) e `txamg202` (Target Tables), insieme ad altre tabelle dedicate al mapping dei campi (`txamg203`).
- **Tracciamento delle Esecuzioni:** lo stato e l'esito delle operazioni, oltre all'eventuale presenza di errori, vengono memorizzati all'interno delle tabelle `txamg300`, `txamg301`, `txamg302` e `txamg303` (Runs Errors).

I Domini includono sia tipi strettamente descrittivi (es. `txamg.cdom`, `txamg.fdnm`) sia tipi enumerativi per la validazione e il workflow (es. `txamg.migt` o `txamg.mapl`).

Esempi di Domini essenziali sono quelli relativi ai formati ASCII (`txamg.fast`), alle regole di mapping (`txamg.mapl`) o al tracciamento dei file (`txamg.dfil`).

Schema ER del modello dati del Migration Tool (Figura 3.1):

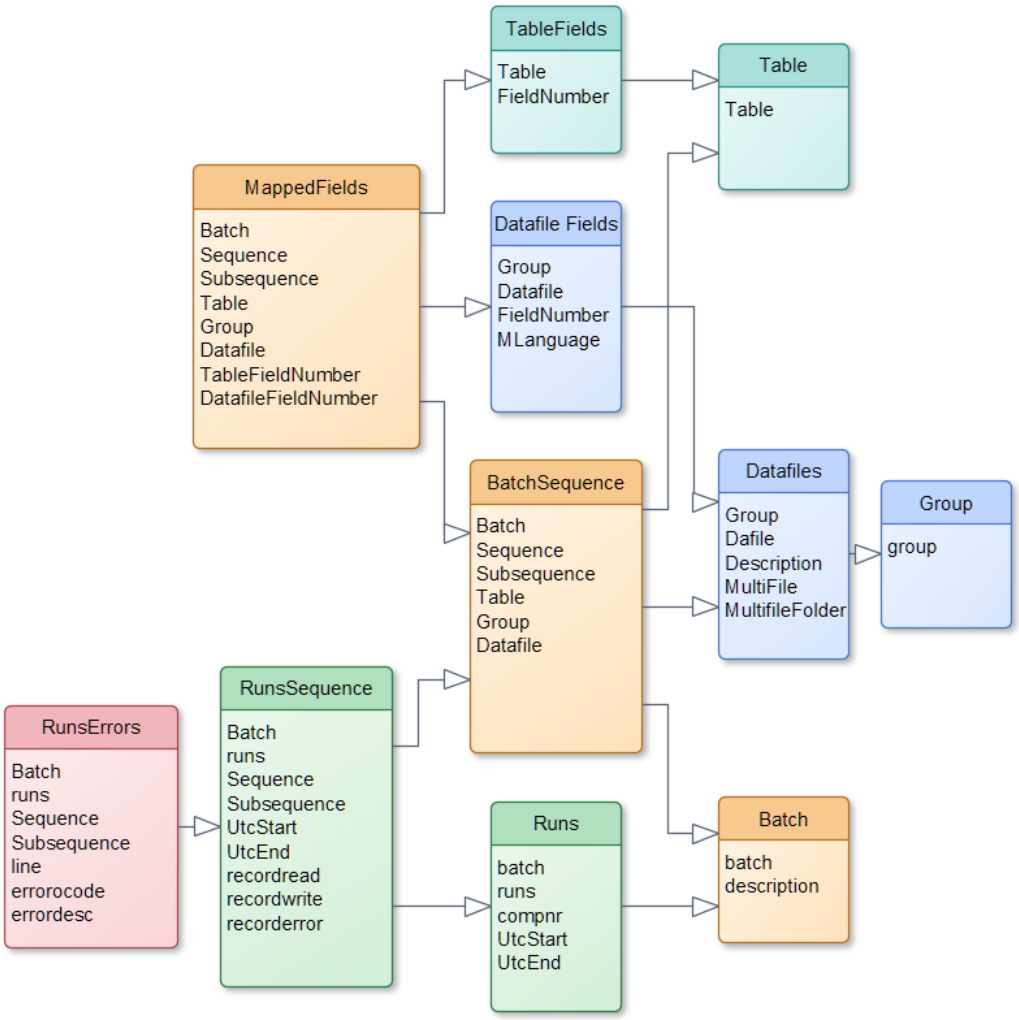



Figura 3.1: Schema ER del Modello Dati del Migration Tool

Tabella delle tabelle principali del Migration Tool:

| **Tabella** | **Nome logico** | **Funzione** |
|---|---|---|
| `txamg000` | AlteaIN Migration Tool Parameters | Parametri globali e configurazione generale del tool |
| `txamg100` | Table groups | Parametri specifici per i batch di migrazione |
| `txamg110` | Datafiles | Definizione dei file di input |
| `txamg111` | DataFile Fields | Dettagli dei campi dei file di input |

Continua nella pagina successiva

| Tabella | Nome logico | Funzione |
|---|---|---|
| txamg200 | Batch | Configurazione processi batch di migrazione |
| txamg201 | Batch Sequences | Sequenze e dipendenze tra batch |
| txamg202 | Target Tables | Definizione delle tabelle target in CloudSuite |
| txamg203 | MappedFields | Mapping logico tra campi di origine e destinazione |
| txamg300 | Runs by Batch | Tracciamento delle esecuzioni dei batch |
| txamg301 | Runs by Batch/Datafile | Tracciamento delle esecuzioni per file specifico |
| txamg302 | Runs by Batch/Datafile/Table | Tracciamento delle esecuzioni per tabella specifica |
| txamg303 | Runs Errors | Dettaglio degli errori riscontrati durante le esecuzioni |

Tabella 3.1: Dizionario delle Tabelle Principali del Migration Tool

Questi elementi compongono le fondamenta dell'integrazione per la generazione del codice di migrazione e dell'importer Infor Exchange.

# Capitolo 4

# Architettura

## 4.1 Panoramica dell'Architettura di Sistema

L'architettura del *Migration Tool* è stata definita per essere modulare e scalabile. Alla base del sistema vi è una netta separazione tra il modulo di *input* (*Importer*), il motore per l'elaborazione dei metadati, e il Generatore di Codice. Questo approccio garantisce la distinzione tra il *layout* dei dati preesistenti e la struttura del database Infor *CloudSuite*.

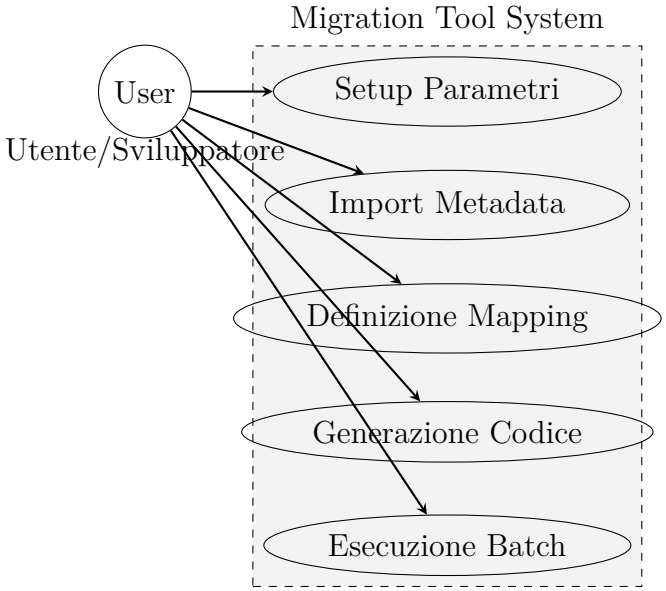



Figura 4.1: Diagramma dei Casi d'Uso del Migration Tool.

### 4.1.1 Modello Dati e Dizionario

L'architettura dati è stata progettata per riflettere e tracciare ogni singolo passaggio del ciclo di vita della migrazione. Il modello relazionale si basa su un raggruppamento logico identificato dai prefissi delle tabelle (sviluppate nel

*package custom* `tx` / modulo `amg`). Le strutture dettagliate di tali tabelle sono consultabili nell'Appendice A.

- **Setup e Parametrizzazione (`txamg000`):** Raccoglie i parametri globali del *tool*, incluse le impostazioni di base per l'I/O (`txamg000.bpat`) e i *package*/*moduli* su cui generare il codice. L'interfaccia di configurazione (Figura 4.2) permette di definire i parametri globali.

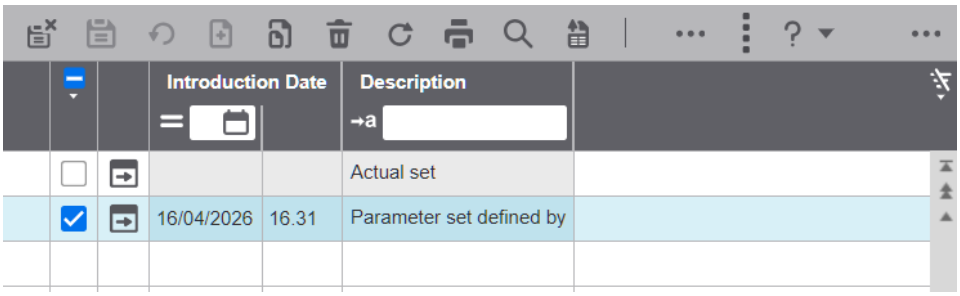



Figura 4.2: Sessione di setup dei parametri globali (`txamg0100m000`).

- **Gestione Metadati (`txamg1xx`):** Le tabelle `txamg110` (*Datafiles*) e `txamg111` (*Datafile Fields*) costituiscono il dizionario della sorgente. Mappano i file di *input*, includendo il formato (es. separatori *ASCII*, lunghezza fissa) e i domini di ciascun campo (tipo di dato, posizioni iniziali e lunghezze).

- **Regole di Mapping (`txamg2xx`):** Rappresentano la parte di logica transazionale. In `txamg202` (*Target Tables*) si definiscono le entità, le logiche di aggiornamento del database e i vincoli di condivisione (*Shared*). Successivamente con `txamg203` (*Mapped Fields*), vengono stabilite le relazioni uno-a-uno e uno-a-molti, standardizzate per le funzioni obbligatorie o opzionali.

- **Esecuzione e Log (`txamg3xx`):** Il sistema adotta una politica di esecuzione strutturata in *run* (elaborazioni). Attraverso `txamg300`, `txamg301` e `txamg302` viene memorizzato il conteggio esatto dei record letti, processati o ignorati in base a date e sequenze. Eventuali criticità e troncamenti vengono inseriti sulla tabella `txamg303` (*Run Errors*).

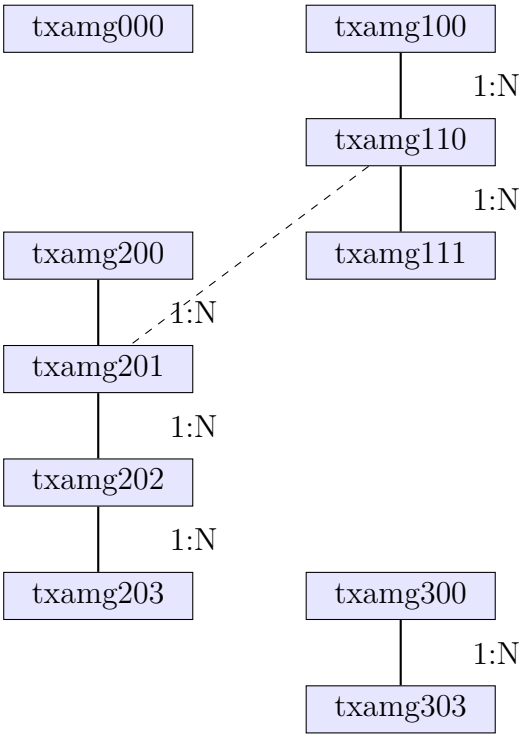


Figura 4.3: Diagramma ER semplificato del *package* txamg.

La Tabella A.1 nell'appendice mostra i parametri di configurazione per gestire i percorsi di I/O e le librerie dinamiche.

## 4.2 Progettazione del Software

### 4.2.1 Design dell'Importer e del Parser dei Dati

L'Importer funge da punto d'ingresso per i dati ed è progettato per interpretare il formato proprietario di Infor *Exchange*, analizzando file di esportazione testuali o strutturati. Estraendo i metadati dai record, l'Importer popola i dizionari *Datafiles* (`txamg110`) e i relativi *Fields* (`txamg111`), la cui struttura è definita rispettivamente nelle tabelle A.3 e A.4.

Questo *design* del caricamento rende possibile supportare sia architetture *multitenant cloud*, sia installazioni *legacy on-premise*.

### 4.2.2 Generazione del Codice

Il generatore di codice è basato su librerie compilate (*DLL* come `txamgdll0012` e `txamgdll0013`), il modulo interpreta il dizionario di *mapping* (definito in `txamg203` (*MappedFields*)) e traduce i legami e le regole di trasformazione in *script* nativi Baan C.

Questo meccanismo previene di dover scrivere manualmente codice, garantendo il rispetto del *framework CloudSuite*: il codice viene prima generato, poi compilato via `bic_compiler`, ed infine agganciato ai *Batch Runs*.

### 4.2.3 Modulo di Integrazione Ad-hoc e Callbacks

Mentre buona parte dei flussi vengono standardizzati automaticamente, alcune regole di *business* richiedono sviluppi specifici a causa della loro complessità. Vengono quindi esposte a livello architetturale delle interfacce di *callback*.

In pratica, in appositi *module script*, gli sviluppatori possono inserire blocchi di codice chiamati asincronamente dal motore generatore di codice e dall'esecutore. Questi *script* personalizzati usufruiscono degli stessi vantaggi transazionali dell'utility principale.

## 4.3 Scelte Tecnologiche

### 4.3.1 Linguaggi e Framework

L'intero ecosistema è sviluppato sfruttando il *Framework* Infor Tools, programmando in **Baan C (sia 3GL che 4GL)**. In parte sono state adottate le *DAL (Data Access Layer)*, per la parte restante, sono state utilizzati approcci più classici e a basso livello.

### 4.3.2 Librerie di Supporto e Strumenti di Build

Tutto il coordinamento della configurazione, *logging* (`txamgdll0011`) e le *string utility* si basano sulle librerie del sistema *ERP* (*DLL* native Baan e *tool* come il `ttdll string`).

Le *build* non prevedono *toolchain* esterni complessi (come *Maven* o *MSBuild*), ma si appoggiano direttamente al *toolset* interno dell'*ERP*, il quale gesti-

sce i legami *VRC* (*Version Release Customer*) *multitenant* per garantirne l'incapsulamento del codice generato al corretto *Tenant*.

# Capitolo 5

# Implementazione

## 5.1 Sviluppo dell'Importer Infor Exchange

### 5.1.1 Gestione dei Formati e Parsing

L'importer è stato sviluppato come un modulo d'*input* che si interpone tra i file d'ingresso (prodotti dai vecchi *server* o estratti tramite *tool legacy*, a volte anche Access) e le tabelle di definizione del nuovo sistema.

La libreria dedicata alla gestione dei *metadata* (`txamgdll0011`) prende in carico l'interpretazione dei metadati (quali formati delimitati o logiche *ASCII* a lunghezza fissa) eseguendo conversioni su base parametrica secondo l'enumerato `txamg.fast`.

Ogni riga recuperata viene controllata per garantire la conformità delle codifiche testo e i troncamenti (*trim*) o *padding* dei campi vuoti.

```
function void GeneraImportUnpack(domain txamg.dfil datafile)
{
    |* Generazione dinamica della scansione della riga ASCII
    |* print_format conterrà una stringa tipo "|%s|%s|%s|"
    |* string_list conterrà la lista delle variabili di destinazione

    select txamg111.* from txamg111
    where txamg111._index1 = {:txamg110.grou, :txamg110.dfil}
    order by txamg111.fdno
    selectdo
        print_format = print_format & first.pipe & "%s|"
        string_list = string_list & comma & "imp.str."
                      & trim$(txamg111.fdnm)
        comma = ","
        first.pipe = ""
    endselect

    Riga("function extern long unpack."&datafile&"(...) {")
    Riga("    ret = string.scan(buffer_"
              &trim$(datafile)&","""
              &trim$(print_format)&""","&trim$(string_list)&")")
    Riga("    ...")
    Riga("}")
}
```

Listing 1: Esempio di generazione del codice per il *parsing* di stringhe *ASCII* (da `txamgdll0012`).

In questo estratto viene mostrato come il generatore iteri sui campi definiti per il *datafile* (`txamg111`) per costruire una chiamata a `string.scan`. Questa funzione Baan C viene poi usata nello *script* generato per «spacchettare» la riga letta dal file *ASCII* nelle variabili corrispondenti ai campi.

### 5.1.2 Ricostruzione delle Definizioni dei File da Migrare

L'obiettivo secondario dell'importer è auto-compilare il *setup* dello stesso *Migration Tool*. Utilizzando le definizioni originarie dei file (le tabelle *legacy*), il programma crea dinamicamente o aggiorna una mappatura interna dei *Data Files* attraverso Sessioni *back-end* come `txamg1110m000`.

Le descrizioni, numerazioni dei campi (`txamg.fdno`) ed i tipi di dominio correlati (`txamg.atyp`) vengono riversati nella tabella di supporto `txamg111` e sono quindi disponibili e precompilati per le successive attività di *data-mapping* manuale verso le tabelle *target CloudSuite*.

L'utente può gestire i gruppi di file ed i singoli file sorgente tramite le sessioni mostrate in Figura 5.1 e 5.2.

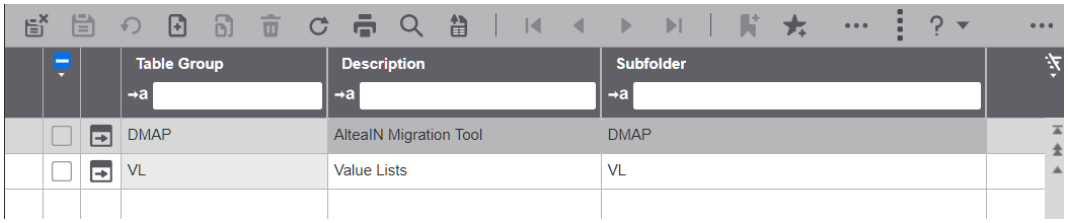



Figura 5.1: DataFile Groups (`txamg1100m000`).

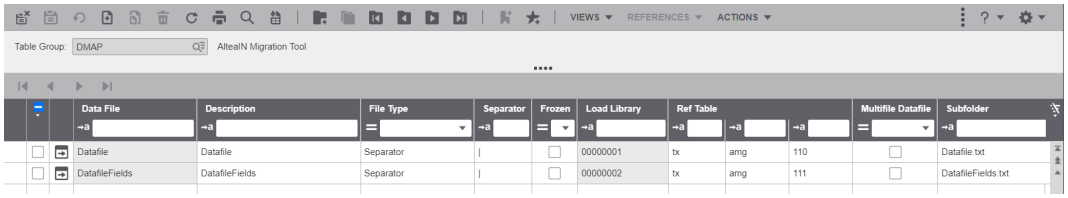



Figura 5.2: DataFiles (`txamg1110m000`).

I dettagli dei campi per ogni file, inclusi i domini e le lunghezze, è gestito in una sessione specifica (Figura 5.3).

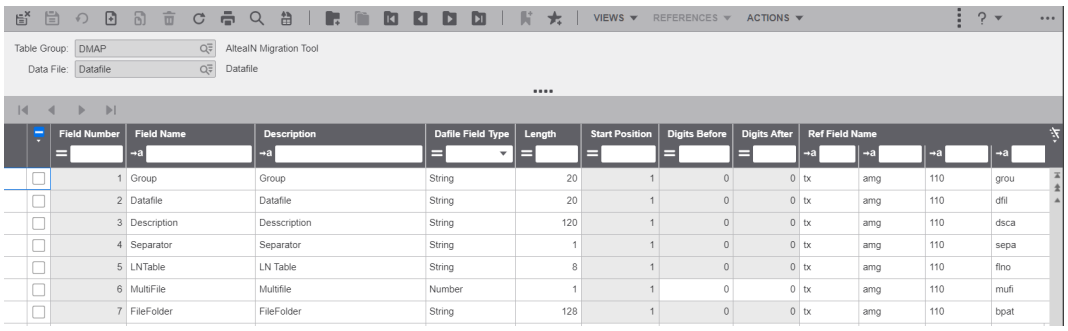



Figura 5.3: Datafile Fields (`txamg1111m000`).

## 5.2 Implementazione del Generatore di Codice

### 5.2.1 Creazione Dinamica del Codice

La generazione dinamica del codice è gestita mediante apposite librerie *DLL* (*Dynamic Link Library*) fornite nel sistema Infor, quali `txamgdll0012` (*Script Generator - Load dll*) e `txamgdll0013` (*Script Generator - Process dll*).

Il codice sorgente Baan C viene generato riga per riga, definendo le firme delle funzioni (es. `function extern long process.import.file...()`) e le variabili necessarie. Per farlo in modo conforme all'infrastruttura di Infor LN, il generatore interroga direttamente il *Data Dictionary* del sistema tramite le tabelle di sistema `ttadv` (in particolare `tttadv420` per le definizioni delle tabelle, `tttadv422` per i campi e `tttadv400` per i domini). Questa interrogazione è delegata al componente `txamgdll0010` (*DD Integration Library*), che funge da strato di astrazione estraendo in automatico tipi di dato, lunghezze massime e *array* per assicurare che lo *script* rispetti le definizioni fisiche corrette.

Tali librerie servono ad analizzare la configurazione definita nelle tabelle di *mapping* (ad esempio `txamg203`) per generare script eseguibili, riducendo drasticamente il tempo manuale richiesto per configurare le regole di conversione standard e garantendo che il codice finale aderisca alla logica di business (*business logic*).

```
1  function void GeneraMap(domain tcmcs.str8 tabella)
2  {
3      Riga("function long map."&tabella&".record() {")
4      select txamg203.*, txamg111.* from txamg203, txamg111
5      where txamg203._index1 =
6      {:txamg202.cbat, :txamg202.sequ, :txamg202.sseq}
7      and txamg203.cmbd refers to txamg111
8      selectdo
9          on case txamg203.type
10             case txamg.type.long:
11                 Riga("    "&tabella&trim$(txamg203.fdnm)
12                          &" = lval(imp.str."&trim$(txamg203.fdnm)&")")
13                 break
14             case txamg.type.string:
15                 Riga("    tt.align.according.domain(imp.str."
16                         &trim$(txamg111.fdnm)&","
17                         &tabella&trim$(txamg203.fdnm)&",...)")
18                 break
19             |* Altri tipi (date, enum, etc.)
20         endcase
21     endselect
22     Riga("    return(OK)")
23     Riga("}")
24 }
```

Listing 2: Logica di generazione della funzione di mapping (da `txamgdll0013`).

Nel seguente esempio viene mostrata la funzione `GeneraMap` che genera il codice per mappare i dati estratti dal file alla struttura della tabella target:

- **Riga 4-6:** Esegue un ciclo sui campi mappati (`txamg203`) relazionandoli con i campi sorgente (`txamg111`).
- **Riga 9-10:** Se il target è un numero intero (*long*), genera una chiamata a `lval()` per convertire la stringa ASCII in valore numerico.
- **Riga 12-13:** Se il target è una stringa, usa `tt.align.according.domain()` per gestire correttamente lunghezze e allineamenti secondo il dominio Infor LN.

### 5.2.2 Funzioni di Trasformazione Ad-hoc

Oltre alle regole predefinite, l'interprete ammette la dichiarazione e l'aggancio di funzioni custom. Durante il processo, eventuali routine di validazione o pulizia del dato possono essere inoltrate ad altre interfacce di esecuzione (come `txamg3200` - Execute Batch Runs).

L'ausilio del componente `txamgdll0011` è importante in quanto si occupa del management dei metadati e dei log contenenti eventuali errori (Error Logging).

## 5.3 Interfacce Utente (Sessioni)

L'interazione con l'utente avviene attraverso un gruppo di sessioni dedicate, denominate *m-sessions* o *s-sessions* a seconda della loro finalità operativa.

Le sessioni principali per la gestione dei `batch` e dei `mapping` sono visualizzabili nelle Figure 5.4, 5.5, 5.6, 5.7.

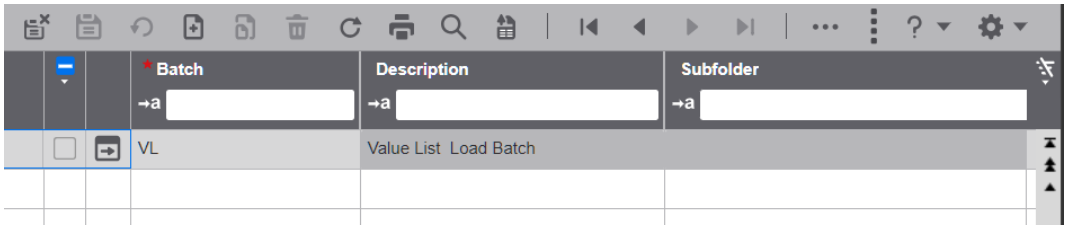



Figura 5.4: Batches (`txamg2100m000`).

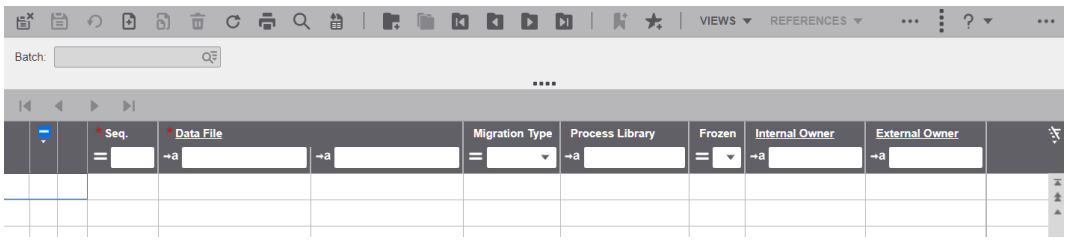



Figura 5.5: Batch Sequences (`txamg2101m000`).

Nelle sessioni dei mapping si definiscono le tabelle target e le logiche di conversione per singolo campo.

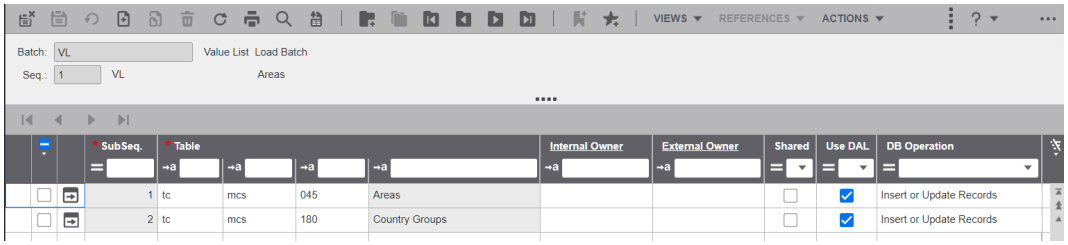



Figura 5.6: Batch Target Tables (`txamg2102m000`).

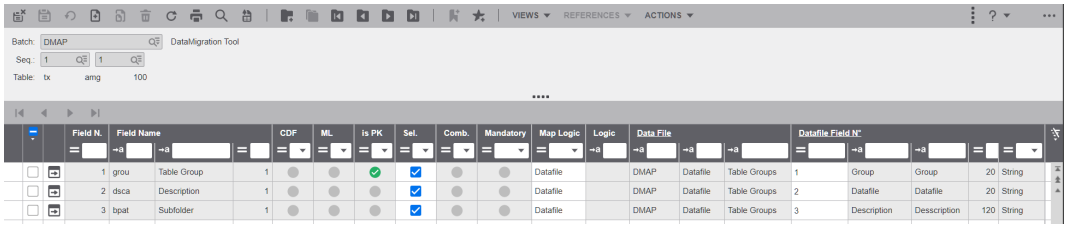



Figura 5.7: Table Mappings (`txamg2103m000`).

Si evidenziano, per esempio:

- **Setup iniziale:** la Sessione `txamg0100m000` permette la parametrizzazione globale dello strumento.
- **Definizione Formati:** le Sessioni relative a `txamg1110` gestiscono il popolamento dei formati dati elaborati.
- **Mapping Dati:** la definizione di conversione logica e associazioni campo-campo tra origine e destinazione avviene nelle interfacce di visualizzazione delle tabelle *target* (`txamg2102m000`) e *fields mapping* (`txamg2103m000`).

Nelle seguenti sezioni vengono riportati gli elenchi dei componenti software sviluppati, suddivisi per tipologia.

### 5.3.1 Esecuzione e Monitoraggio

Una volta configurati i mapping e generato il codice, il tool permette l'esecuzione dei batch tramite la sessione `txamg3200m000` (Figura 5.8).




Figura 5.8: Execute Batch Runs (`txamg3200m000`).

Al termine dell'esecuzione, è possibile consultare alcune statistiche come momento di inizio e di fine e i log d'errore per ogni run (Figura 5.9 e 5.10).

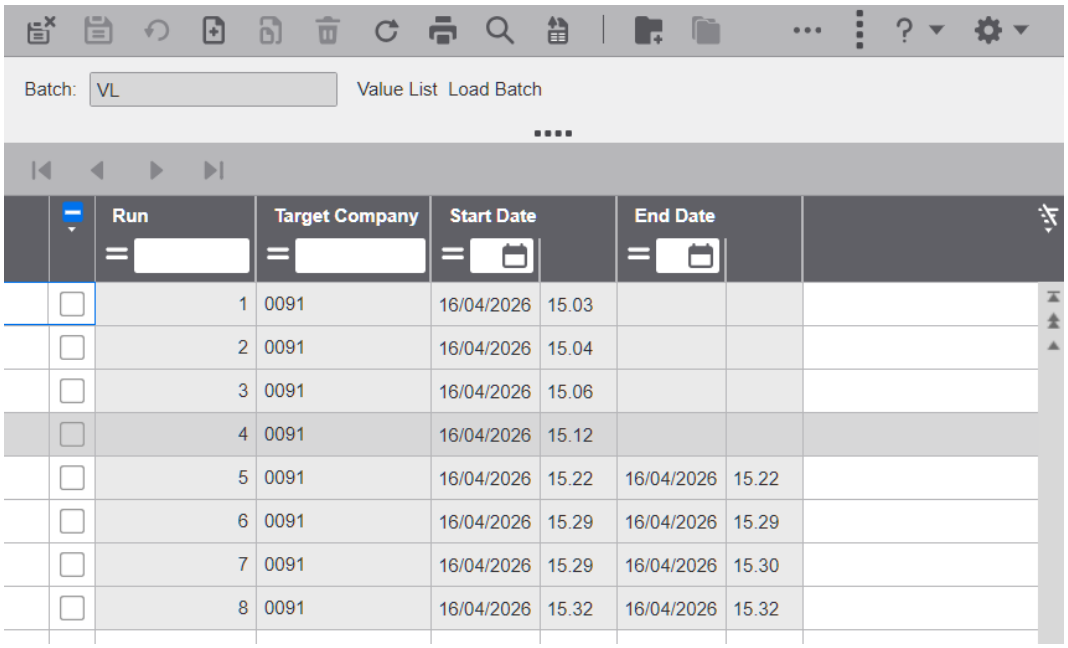



Figura 5.9: Runs By Batch (`txamg3500m000`).

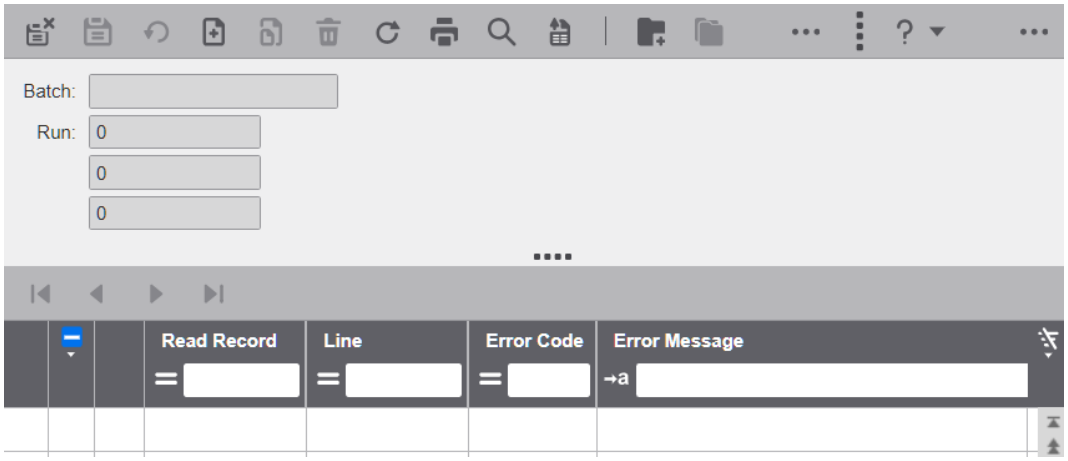



Figura 5.10: Runs Errors (`txamg3503m000`).

### 5.3.2 Librerie (DLL) e Data Access Layer (DAL)

| Tipo | Codice | Descrizione |
|---|---|---|
| Libreria | `txamgdll0010` | AlteaIN Migration Tool - DD Integration Library |
| Libreria | `txamgdll0011` | AlteaIN Migration Tool - Metadata Management & Error Logging |
| Libreria | `txamgdll0012` | AlteaIN Migration Tool - Script Generator - Load DLL |
| Libreria | `txamgdll0013` | AlteaIN Migration Tool - Script Generator - Process DLL |
| DAL | `txamg000` | DAL for txamg000 |
| DAL | `txamg111` | DAL for txamg111 |
| DAL | `txamg203` | DAL for txamg203 |
| DAL | `txamg1110` | Data Files |
| DAL | `txamg3200` | Execute Batch Runs |
| DAL | `txamg3500` | Runs by Batch |
| DAL | `txamg3501` | Runs by Batch/Datafile |
| DAL | `txamg3502` | Runs by Batch/Datafile/Table |
| DAL | `txamg3503` | Runs Errors |

Tabella 5.1: Elenco delle Librerie (DLL) e DAL sviluppati.

### 5.3.3 Tabelle di Sistema e Sessioni Operative

Elenco delle principali tabelle (le strutture dettagliate sono consultabili nell'Appendice A) e delle sessioni utilizzate.

| Tabella | Descrizione |
|---|---|
| txamg000 | AlteaIN Migration Tool Parameters |
| txamg100 | Table groups definitions |
| txamg110 | Datafiles source definitions |
| txamg111 | DataFile Fields specifications |
| txamg200 | Batch definitions |
| txamg201 | Batch Sequences execution order |
| txamg202 | Target Tables mapping |
| txamg203 | MappedFields specific logic |
| txamg300 | Runs by Batch execution logs |
| txamg301 | Runs by Batch/Datafile detailed logs |
| txamg302 | Runs by Batch/Datafile/Table detailed logs |
| txamg303 | Runs Errors repository |

Tabella 5.2: Riepilogo delle tabelle introdotte dal Migration Tool.

| Sessione | Descrizione |
|---|---|
| txamg0100m000 | Parameters |
| txamg1100m000 | Datafile Groups |
| txamg1110m000 | Data Files |
| txamg1111m000 | DataFile Fields |
| txamg2100m000 | Batches |
| txamg2101m000 | Batch Sequences |
| txamg2102m000 | Batch Target Tables |
| txamg2103m000 | Table Mappings |
| txamg3200m000 | Execute Batch Runs |
| txamg3500m000 | Runs by Batch |
| txamg3503m000 | Runs Errors |

Tabella 5.3: Sessioni principali sviluppate per il Migration Tool.

| Dominio | Descrizione |
|---|---|
| txamg.atyp | DataFile Field Type (Enumerated) |
| txamg.cbat | Batch (String) |
| txamg.cdom | Domain (String) |
| txamg.cmod | Module (String) |
| txamg.cpac | Package (String) |
| txamg.cprs | Script/Library Code (String) |
| txamg.dfil | Data File (String) |
| txamg.dfnm | Data File Field Name (String) |
| txamg.fast | ASCII File Format (Enumerated) |
| txamg.fdnm | Field Name (String) |
| txamg.fdno | Field Number (Integer) |
| txamg.flno | Table (String) |
| txamg.grou | Table Groups (String) |
| txamg.logn | Login (String) |
| txamg.mapl | Map Logic (Enumerated) |
| txamg.migt | Migration Type (Enumerated) |
| txamg.refm | Reference Mode (Enumerated) |
| txamg.type | Domain Type (Enumerated) |
| txamg.utyp | Update Type (Enumerated) |

Tabella 5.4: Domini personalizzati tx del Migration Tool.

## 5.4 Integrazione Documentale

### 5.4.1 Esportazione delle Specifiche di Migrazione

L'integrazione formale del modulo d'implementazione sfocia nella sua interconnessione con lo strumento aziendale adibito alla documentazione del progetto.

Prima della configurazione ed eventuale esecuzione esplorativa del batch (le sessioni `txamg2100` e `txamg3200`), l'*utility* estrae il risultato dei mapping salvato nel `txamg203` (Tabella Mappings), formattandolo utilizzando layout predefiniti.

L'esportazione documenta in PDF ogni singolo dominio trasferito, documentando sia i campi copiati senza modifiche (*one-to-one*) (in accordo con `txamg.mapl`) sia quei casi particolari per cui è stato inserito del codice personalizzato, semplificando una possibile manutenzione logica dell'architettura in futuro.

## 5.5 Ciclo di Vita del Dato

Il processo di migrazione segue un flusso lineare, descritto nel seguente diagramma:

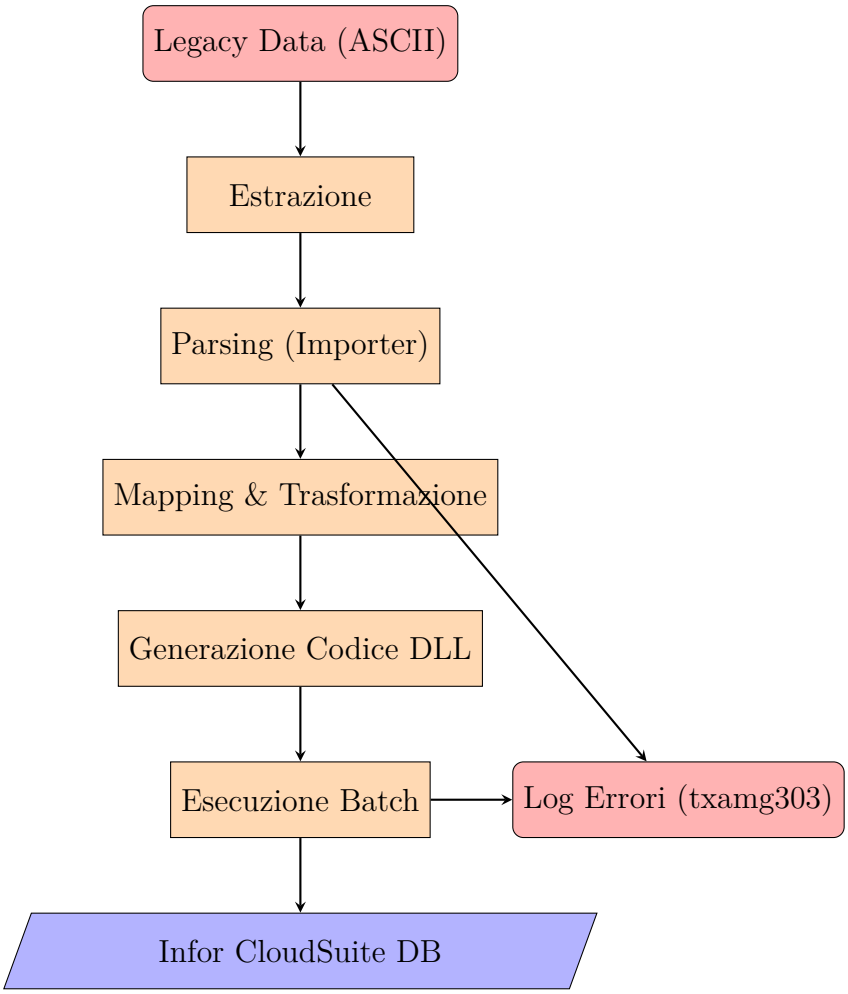



Figura 5.11: Diagramma di Flusso del Ciclo di Vita del Dato.

Il flusso garantisce che ogni record sia tracciato. In caso di errore durante il parsing (formato scorretto) o durante l'esecuzione (vincoli di database), l'engine scrive un record dettagliato nella tabella `txamg303`, permettendo la ripartenza selettiva della migrazione.

# Capitolo 6

# Testing e Validazione

## 6.1 Strategia di Testing

Il testing di un software per la migrazione dei dati è un aspetto fondamentale. Data la sensibilità delle logiche del *Data Dictionary* di Infor CloudSuite in ambienti Cloud Multitenant, ogni anomalia nel record di inserimento avrebbe gravi conseguenze nel sistema target.

Inoltre, per il generatore dinamico di codice Baan C, le procedure di collaudo hanno dovuto coprire non solo l'importer dei dati, ma anche la correttezza sintattica e logica degli script generati a runtime dalle DLL `txamgdll0012` e `txamgdll0013`.

### 6.1.1 Unit Testing sui Moduli di Parsing

I moduli di base (DLL quali `txamgdll0011`) per l'estrazione e la gestione dei metadata (*metadata management*) degli Infor Exchange sono stati collaudati su porzioni specifiche di file.

Verificando ogni singola colonna (`txamg1111`), la validazione include il corretto casting di tipi enumerati, decimali a virgola mobile e codifiche caratteri ASCII, simulando una diversità di input.

### 6.1.2 Test di Integrazione del Flusso di Migrazione

L'integrazione end-to-end è validata eseguendo flussi di test governati dalle logiche di batch (`txamg2100` e `txamg3200`). Durante questi test, il sistema genera gli script di elaborazione e li sottopone al compilatore interno `bic_compiler`. Se la generazione avviene correttamente, lo script compilato viene agganciato al batch e lanciato.

I *Runs Errors* offrono un riscontro immediato di blocchi logici o semantici. Sono stati eseguiti scenari in cui la validazione è forzata per respingere record privi di riferimenti chiave, verificando che la linea incriminata fosse elaborata senza però interromperne il caricamento dell'intero *Run*.

## 6.2 Verifica dell'Integrità dei Dati

### 6.2.1 Matching tra Dati Sorgente (Legacy) e Destinazione (Cloud)

Ogni file migrato è dotato di un tracciamento specifico (`txamg301`, `txamg302`). Il *matching* prevede che vengano comparati il numero dei record letti, processati ed eventualmente respinti rispetto alla fonte.

I report prodotti (tramite moduli ad hoc come `txamgdll0011` per il log d'errore) forniscono la garanzia numerica dell'operazione.

# Capitolo 7

# Conclusioni

## 7.1 Obiettivi Raggiunti e Risultati Ottenuti

Nel percorso di questa tesi, è stato analizzato, implementato e documentato un completo *Migration Tool* (pacchetto aziendale `txamg`) per l'ambiente Infor LN. C'erano requisiti particolarmente sfidanti dietro a questo progetto, nati per la maggior parte dalla natura *multitenant* dei nuovi ambienti *CloudSuite*, che sono stati affrontati offrendo un approccio fortemente parametrico e automatizzato.

Lo sviluppo di librerie capaci di generare dinamicamente codice leggendo il dizionario (come `txamgdll0012` e `txamgdll0013`) e un *importer* capace di decifrare nativamente i formati Infor *Exchange*, ha permesso di centrare in pieno l'obiettivo aziendale richiesto. In questo modo si sono potuti garantire ottimi risultati in termini di affidabilità e *performance*, alleggerendo notevolmente i consulenti dallo snervante e ripetitivo compito della trascrizione manuale, che spesso si traduceva in facili errori.

## 7.2 Sfide Incontrate

Le principali difficoltà che hanno accompagnato tutto il periodo di progetto sono nate dal dover imporre un *mapping* rigoroso, lavorando in gran parte su un sistema chiuso (proprietario Infor), in cui l'utilizzo esclusivo dei linguaggi Baan C (3GL e 4GL) ha rappresentato il vincolo tecnico ineliminabile.

Lavorando in questo ecosistema ristretto, avere la possibilità di interfacciarsi con il compilatore interno (e validare il codice *target* a *runtime* su ogni specifico *tenant*) ha fatto un'enorme differenza dal punto di vista dell'analisi e *debug*. Tuttavia, standardizzare formati eterogenei del mondo *legacy* ha imposto la

creazione di strumenti di *parsing* e configurazione di *Runs batch* piuttosto complessi per il modulo `txamg200`.

Durante lo fase di studio e sviluppo, è stato necessario affrontare una fase di *training* sia base che avanzata dei *tool* Infor, quali struttura, nomenclatura, linguaggi, logica di funzionamento, architettura come *middleware ION*, *BOD*, *API* e *Web Services* con formati *XML legacy* come *SOAP* (non *REST*), in tempi molto brevi rispetto a quanto normalmente previsto (diversi mesi), al fine di completare il progetto in tempo e completare questa tesi.

Si fa anche notare la natura molto restrittiva dell'*ERP* Infor, che non permette l'accesso nemmeno ad amministratori di sistema a parti di *API* o funzionalità che semplificherebbero lo sviluppo, oltre alla mancata documentazione delle stesse in molti casi a causa di transizioni a più alto livello aziendale, lasciando allo sviluppatore finale (noi) «scoprire» ed esplorare tramite *trial-and-error* delle soluzioni, spesso non *standard*, ove possibile. Questo ha dato origine molti tempi morti, complicando il contesto generale

## 7.3 Sviluppi Futuri

Nonostante lo *standard* d'eccellenza raggiunto, si ritiene possibile evolvere ulteriormente il *tool* in diverse direzioni.

In primo luogo, lo schema architetturale per le estensioni *Custom* potrebbe venire affiancato da procedure di *Machine Learning* (addestrate sui *mapping legacy*) in grado di suggerire i *mapping* e i moduli di conversione per grandi volumi di tabelle non ancora definite (senza l'*input* umano in interfacce e Sessioni `txamg2103`). Inoltre, la visualizzazione delle specifiche tramite esportazione a modelli *UML* o grafi digitali, se automatizzata nativamente dal *framework*, fornirebbe reportistica in *real-time* durante la fase di studio del cliente prima dell'operazione di caricamento, riducendo di fatto tempi e costi e supportando i consulenti AlteaIN, funzionali e Infor verso transizioni *Cloud* più semplici.

# Appendice A

# Dettaglio della Struttura delle Tabelle

Viene qui riportata la struttura dettagliata delle tabelle progettate, sviluppate e utilizzate all'interno del Migration Tool. Ogni tabella è identificata da un codice univoco con prefisso `txamg`. Ove le prime due lettere (`tx`) indicano il package *custom*, le successive tre lettere (`amg`) rappresentano il modulo.

## A.1 Tabelle di Configurazione e Metadati

Tabella A.1: Struttura della tabella txamg000 (AlteaIN Migration Tool Parameters)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| indt | tc date | UTC Date/Time | 22 | Introduction Date |
| dsca | tc mcs.str50m | Multibyte String | 50 | Description |
| lopk | tx amg.cpac | String | 2 | Load Library Package |
| lomd | tx amg.cmod | String | 3 | Load Library Module |
| loff | tc mcs.long | Long | 10 | Load Library First Free Number |
| prpk | tx amg.cpac | String | 2 | Process Library Package |
| prmd | tx amg.cmod | String | 3 | Process Library Module |
| prff | tc mcs.long | Long | 10 | Process Library First Free Number |
| bpat | tc path2 | String | 128 | Base Path for Files |

Tabella A.2: Struttura della tabella txamg100 (Table groups)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| grou | tx amg.grou | String | 20 | Table Group |
| dsca | tc dsca | Multibyte String | 30 | Description |

Tabella A.2: Struttura della tabella txamg100 (Table groups) – Continua

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| bpat | tc path2 | String | 128 | Subfolder |

Tabella A.3: Struttura della tabella txamg110 (Datafiles)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| grou | tx amg.grou | String | 20 | Table Group |
| dfil | tx amg.dfil | String | 20 | Data File |
| dsca | tc dsca | Multibyte String | 30 | Description |
| fast | tx amg.fast | Enumerated | 10 | File Type |
| sepa | tc mcs.str1 | String | 1 | Separator |
| frzn | tc yesno | Enumerated | 5 | Frozen |
| cprs | tx amg.cprs | String | 9 | Load Library |
| cpac | tx amg.cpac | String | 2 | Ref Table Package |
| cmod | tx amg.cmod | String | 3 | Ref Table Module |
| flno | tx amg.flno | String | 3 | Ref Table |
| mufi | tc yesno | Enumerated | 5 | Multifile Datafile |
| bpat | tc path2 | String | 128 | Subfolder |

Tabella A.4: Struttura della tabella txamg111 (DataFile Fields)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| grou | tx amg.grou | String | 20 | Table Group |
| dfil | tx amg.dfil | String | 20 | Data File |
| fdno | tx amg.fdno | Integer | 4 | Field Number |
| fdnm | tx amg.dfnm | String | 20 | Field Name |
| dsca | tc dsca | Multibyte String | 30 | Description |
| stpn | tx amg.fdno | Integer | 4 | Start Position |
| leng | tx amg.fdno | Integer | 4 | Length |
| atyp | tx amg.atyp | Enumerated | 10 | Dafile Field Type |
| adgb | tc mcs.int | Integer | 3 | Digits Before |
| adga | tc mcs.int | Integer | 3 | Digits After |
| cpac | tx amg.cpac | String | 2 | Ref Table Package |
| cmod | tx amg.cmod | String | 3 | Ref Table Module |

Tabella A.4: Struttura della tabella txamg111 (DataFile Fields) – Continua

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| flno | tx amg.flno | String | 3 | Ref Table |
| rfnm | tx amg.fdnm | String | 8 | Ref Field Name |

## A.2 Tabelle di Mapping e Logica

Tabella A.5: Struttura della tabella txamg200 (Batch)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| cbat | tx amg.cbat | String | 20 | Batch |
| dsca | tc dsca | Multibyte String | 30 | Description |
| bpat | tc path2 | String | 128 | Subfolder |

Tabella A.6: Struttura della tabella txamg201 (Batch Sequences)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| cbat | tx amg.cbat | String | 20 | Batch |
| sequ | tc mcs.int | Integer | 3 | Seq. |
| grou | tx amg.grou | String | 20 | Table Group |
| dfil | tx amg.dfil | String | 20 | Data File |
| migt | tx amg.migt | Enumerated | 10 | Migration Type |
| cprs | tx amg.cprs | String | 9 | Process Library |
| ilgn | tx amg.logn | String | 8 | Internal Owner |
| elgn | tx amg.logn | String | 8 | External Owner |
| txta | tc txtn | Text | 0 | Notes |
| frzn | tc yesno | Enumerated | 5 | Frozen |

Tabella A.7: Struttura della tabella txamg202 (Target Tables)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| cbat | tx amg.cbat | String | 20 | Batch |
| sequ | tc mcs.int | Integer | 3 | Seq. |
| sseq | tc mcs.int | Integer | 3 | SubSeq. |
| cpac | tx amg.cpac | String | 2 | Package |

Tabella A.7: Struttura della tabella txamg202 (Target Tables) – Continua

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| cmod | tx amg.cmod | String | 3 | Module |
| flno | tx amg.flno | String | 3 | Table |
| flnm | tc dsca | Multibyte String | 30 | Table Name |
| ilgn | tx amg.logn | String | 8 | Internal Owner |
| elgn | tx amg.logn | String | 8 | External Owner |
| shrd | tc yesno | Enumerated | 5 | Shared |
| udal | tc yesno | Enumerated | 5 | Use DAL |
| utyp | tx amg.utyp | Enumerated | 10 | DB Operation |
| txta | tc txtn | Text | 0 | Functional Notes |

Tabella A.8: Struttura della tabella txamg203 (Mapped Fields)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| cbat | tx amg.cbat | String | 20 | Batch |
| sequ | tc mcs.int | Integer | 3 | Seq. |
| sseq | tc mcs.int | Integer | 3 | SubSeq. |
| cpac | tx amg.cpac | String | 2 | Package |
| cmod | tx amg.cmod | String | 3 | Module |
| flno | tx amg.flno | String | 3 | Table |
| fdno | tx amg.fdno | Integer | 4 | Field N. |
| fdnm | tx amg.fdnm | String | 8 | Field Name |
| icdf | tc yesno | Enumerated | 5 | CDF |
| isml | tc yesno | Enumerated | 5 | ML |
| fdsc | tc dsca | Multibyte String | 30 | Field Desc |
| comb | tc yesno | Enumerated | 5 | Comb. |
| pacd | tx amg.cpac | String | 2 | Domain Package |
| cdom | tx amg.cdom | String | 12 | Domain |
| type | tx amg.type | Enumerated | 10 | Data Type |
| digb | tc mcs.int | Integer | 3 | Digit Before |
| diga | tc mcs.int | Integer | 3 | Digit After |
| iniv | tc dsca | Multibyte String | 30 | Initial Value |
| mand | tc yesno | Enumerated | 5 | Mandatory |
| rfpa | tx amg.cpac | String | 2 | Ref. Table Package |

Tabella A.8: Struttura della tabella txamg203 (Mapped Fields) – Continua

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| rfmo | tx amg.cmod | String | 3 | Ref. Table Module |
| rffn | tx amg.cmod | String | 3 | Ref. Table |
| rffd | tx amg.fdnm | String | 8 | Ref. Field |
| dept | tc mcs.int | Integer | 3 | Depth |

## A.3 Tabelle di Esecuzione e Log

Tabella A.9: Struttura della tabella txamg300 (Runs by Batch)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| cbat | tx amg.cbat | String | 20 | Batch |
| runs | tc mcs.long | Long | 10 | Run |
| ncmp | tc ncmp | Integer | 4 | Target Company |
| stdt | tc date | UTC Date/Time | 22 | Start Date |
| endt | tc date | UTC Date/Time | 22 | End Date |

Tabella A.10: Struttura della tabella txamg301 (Runs by Batch/Datafile)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| cbat | tx amg.cbat | String | 20 | Batch |
| runs | tc mcs.long | Long | 10 | Run |
| sequ | tc mcs.long | Long | 10 | Sequence |
| stdt | tc date | UTC Date/Time | 22 | Start Date |
| endt | tc date | UTC Date/Time | 22 | End Date |
| rere | tc mcs.long | Long | 10 | Records Read |
| repr | tc mcs.long | Long | 10 | Record Processed |
| reer | tc mcs.long | Long | 10 | Records Discarded |

Tabella A.11: Struttura della tabella txamg302 (Runs by Batch/Datafile/Table)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| cbat | tx amg.cbat | String | 20 | Batch |
| runs | tc mcs.long | Long | 10 | Run |

Tabella A.11: Struttura della tabella txamg302 (Runs by Batch/Datafile/Table) – Continua

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| sequ | tc mcs.long | Long | 10 | Sequence |
| sseq | tc mcs.long | Long | 10 | Subsequence |
| stdt | tc date | UTC Date/Time | 22 | Start Date |
| endt | tc date | UTC Date/Time | 22 | End Date |

Tabella A.12: Struttura della tabella txamg303 (Runs Errors)

| Campo | Dominio | Tipo Dato | Lung. | Descrizione |
|---|---|---|---|---|
| cbat | tx amg.cbat | String | 20 | Batch |
| runs | tc mcs.long | Long | 10 | Run |
| sequ | tc mcs.long | Long | 10 | Sequence |
| sseq | tc mcs.long | Long | 10 | Subsequence |
| rere | tc mcs.long | Long | 10 | Read Record |
| line | tc mcs.long | Long | 10 | Line |
| errc | tc mcs.int | Integer | 3 | Error Code |
| errm | tc mcs.s999m | Multibyte String | 999 | Error Message |

# Bibliografia


[1] Y. Guo et al. «Multi-tenant cloud computing architecture: A review». In: *Journal of Ambient Intelligence and Humanized Computing* (2020) (cit. alle pp. 1, 5).

[2] *Infor Cloudsuite Enterprise*. Accessed: 2026-04-16. 2026. URL: `https://www.infor.com/products/cloudsuite-enterprise` (cit. alle pp. 1, 6).

[3] Infor Development Team. *Infor LN - 3GL and 4GL Programming Guide*. Manuale per lo sviluppo di script e personalizzazioni in linguaggio Baan C. Infor. 2025 (cit. a p. 8).

[4] Infor Development Team. *Infor LN - Data Access Layer (DAL) Overview and Reference*. Documentazione tecnica per l'implementazione della logica di business e integrità dati. Infor. 2025 (cit. a p. 7).

[5] Infor Development Team. *Infor LN - Extended Data Access Layer (DAL2) Overview*. Documentazione tecnica sull'evoluzione del framework DAL per l'integrazione e il 4GL. Infor. 2025 (cit. a p. 8).

[6] *Infor LN - Cloud ERP for Manufacturing*. Accessed: 2026-04-16. 2026. URL: `https://www.infor.com/products/ln` (cit. alle pp. 1, 6).

[7] B. Johansson e F. Sudzina. «Challenges in Data Migration to ERP Systems». In: *Enterprise Information Systems* (2008) (cit. alle pp. 5, 11).

[8] Johny Morris. *Practical Data Migration*. BCS, The Chartered Institute for IT, 2012 (cit. alle pp. 6, 11).

[9] A. Verazzi, A. Cerini e J. P. Franceschetti. *Infor LN - Exchange Guide (Revision 02)*. Guida alla configurazione degli schemi di importazione ed esportazione ASCII. Infor. 2025 (cit. a p. 8).